

区块链技术与应用（2）——以太坊（1）：账户与数据结构

以太坊（ETH）概述

以太坊也被称之为Blockchain2.0

以太坊之于区块链的区别与优化

时间

以太坊产生新区块的时间只需要十几秒，大大加快了交易发布的效率

而基于我们在上一章“挖矿”里面讲到的，为什么要限制挖矿速率的内容。

我们很容易想到，ETH这样的速度如果按照BTC的共识机制，很容易分叉并造成其他问题。

对此，以太坊设计了Goast共识机制，以后会讲。

算力

以太坊设计了限制ASIC芯片的挖矿机制，加深了去中心化程度

以太坊设计的mining puzzle，对于内存有较高要求，因此具有天然的ASIC resistance特性

这样的话就放置类BTC那样的算力专业化效果（矿池的出现）

权益证明

ETH设计了proof of stake（股份投票），不同于BTC的PoW

智能合约

智能合约（smart contract），实现了合约的去中心化

通常，货币和合约是政府以司法手段维持的体系。

比特币实现的是货币的去中心化，以太坊开创了合约的去中心化。

合约的去中心化带来了如下的好处：

1. 在无统一司法管辖的个体之间依旧可以实现合约
2. 合约从缔结开始就无法更改
3. 避免了司法手段长周期带来的麻烦

局限：

1. 涉及的合约内容需要代码实现，需要合约内容相对简洁

货币

BTC货币体系：

基本单位：比特币（BTC）

最小单位：聪（Satoshi）100,000,000

ETH货币体系：

基本单位：以太币（Ether）

最小单位：维（wei）1,000,000,000,000,000,000

账户

比特币的账户模式是与现实更为接近的 账户余额模式（account_based_ledger）

与BTC相比的优化

账户的显示化

使得用户使用时能够更清楚的得知自己的账户余额

资金来源

相比于BTC每次交易都需要查找货币来源（维持UTXO模型），ETH使用的账户模式直观的展现一个账户的现有金额，账户上的每一维钱都是合法的。

这也催生了下面的特性

天然防双花性

因为账户账目一目了然，够不够花只用简单的比大小就行，因此天然的防止了双花的出现

花费零细化

BTC一个很反常识的操作就是账户上的钱必须花完

这样就导致，支付交易的账户可能有多个，接受交易的账户也可能有多个（接收找零）

一个人创建多个账户，账户很肯能就做一次转账，这样很容易影响账户的稳定性，进而影响合约。

而ETH这样做简化了交易，而且维持了账户的稳定

账户分类

外部账户（普通账户）

这些账户和BTC里面的轻节点账户很类似

账户也是由公私钥对创建的，账户的地址是公钥的哈希值（取后四十位）

他们只有：账户余额（balance），从创建账户开始到现在的交易次数（nonce）

合约账户（smart contract account）

合约账户保存的数据结构：nonce（合约从创建开始至今被调用的次数），code（合约的代码），storage（变量）

以太坊合约账户的storage包含合约中定义的所有状态变量。这些状态变量记录了合约的状态和数据，可以被读取或修改。在以太坊上，存储在合约账户中的状态变量是永久性存储，即它们会持久保存在区块链上，可以被所有参与者查看和验证。

账户使用规定

1. 交易的发起者只能是外部账户
2. 一个合约账户，在交易的过程中，可以通过发送message来调用其他合约账户
3. 创建合约账户时会返回一个地址，通过这个地址来调用合约（这时storage会变）

数据结构

ETH中最优秀的三个数据结构就是它的三棵树

状态树

我们思考一个问题，ETH的数据结构相对于BTC究竟需要实现那些应用上的差别？

1. **包含信息**，BTC里面包含的是新增的几百个交易，而ETH需要的是所有的账户信息
2. **产生时间**，BTC的出块时间是10min，ETH是几十秒，这就使得ETH的数据结构需要简洁高效的组织形式

从BTC出发，分析为什么要用MPT

对于键值对的查找，我们首先想到的就是哈希表。

为什么哈希表不行？

首先，哈希表这个东西就无法实现Merkle Proof（这种证明是轻节点非常需要的确保自己账户安全的工具）

再者，为了维护全节点的一致性，必须给出一个一致的默尔克树根哈希值，这也是哈希表无法实现的

那么我们能不能在生成哈希表后，构建一个默尔克树呢？

这很显然不现实！

因为BTC中，一个区块生成需要十分钟，里面顶多包含4000个交易，很好构建

但是ETH要想构建，前提就要把所有账户的信息全包含进哈希表中去，并且此后构建默尔克树。该过程只有十几秒时间

另外，在这十几秒中，只有很少一部分账户是发生改变的，这就使得上述的方法更没必要了。

那么我们能不能使用默尔克树呢？

首先，正常默尔克树是无法实现高效的查找的（只能一个一个找）

再者，每个全节点构建的默尔克树中的账户顺序是不一样的，这会使根哈希值不一样，没法通过验证根哈希值来维持系统账户的一致性

那么我们能不能使用sorted merkle tree？

新增账户问题！由于哈希值的计算生成的字符串不是由大到小的，因此我们没法实现**按顺序的插入**

那么每次插入新的账户就需要重构这个Merkle Tree

最后，就是大多账户的状态都是不变的，那么生成所有账户信息的数据结构就显得很傻

基于字典树（trie）的数据结构设计

需要怎样的数据结构？

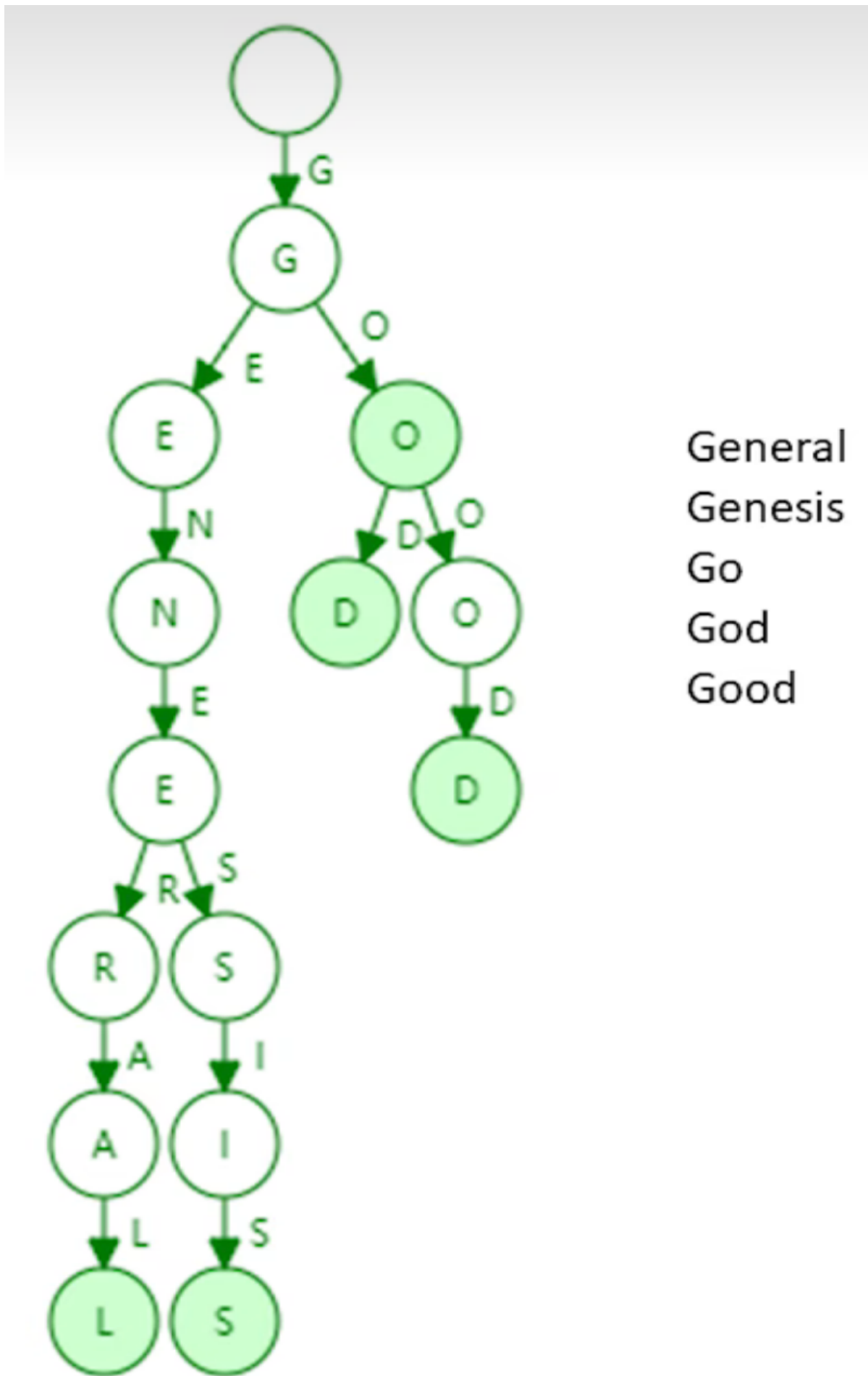
借鉴了BTC的经验，我们发现它的哈希表和默尔克树都不太合适。

在上述讨论中，我们不难发现，为了满足ETH，我们需要的数据结构要满足一下特征：

便于插入，便于查找，能提供MerProof，能给出一致的根哈希值

下面，我们通过实例认识一下字典树！

我们拿5个单词举例



我们可以看出，这个数据结构既解决了排序的问题，方便查找每一个元素；也方便插入元素。这种数据结构被称为trie

字典树

字典树的特点

1. 分叉的数目最多是元素取值范围+1。如果是26个字母，就有27种分叉（还有一个结束标志位）；如果是我们的16进制哈希值，那么最多17个分叉
2. 查找元素的效率取决于链的长度

字典树的优点

1. 顺序是固定的
2. 更新是局部的

字典树的不足

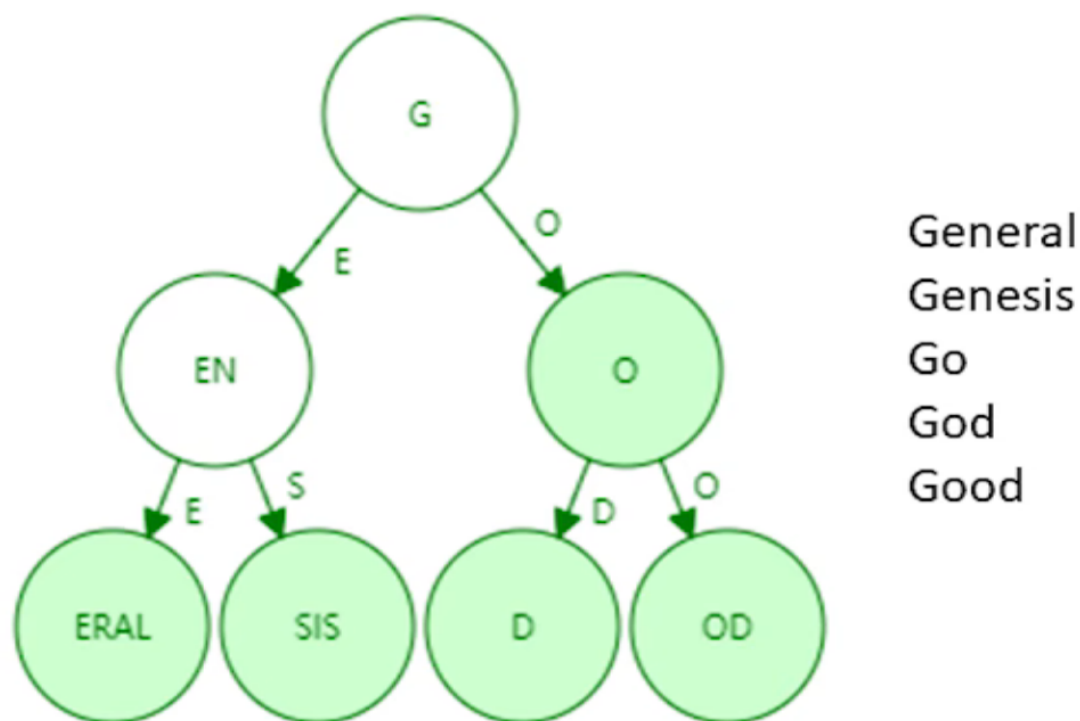
1. 链长，查找效率低
2. 存储密度低，浪费

因此，我们设计了下面的数据结构

Modified trie

这个数据结构相较于字典树的优化是：**缩短了每条链的长度**

我们想一想，在什么情况下适用下面的结构呐？

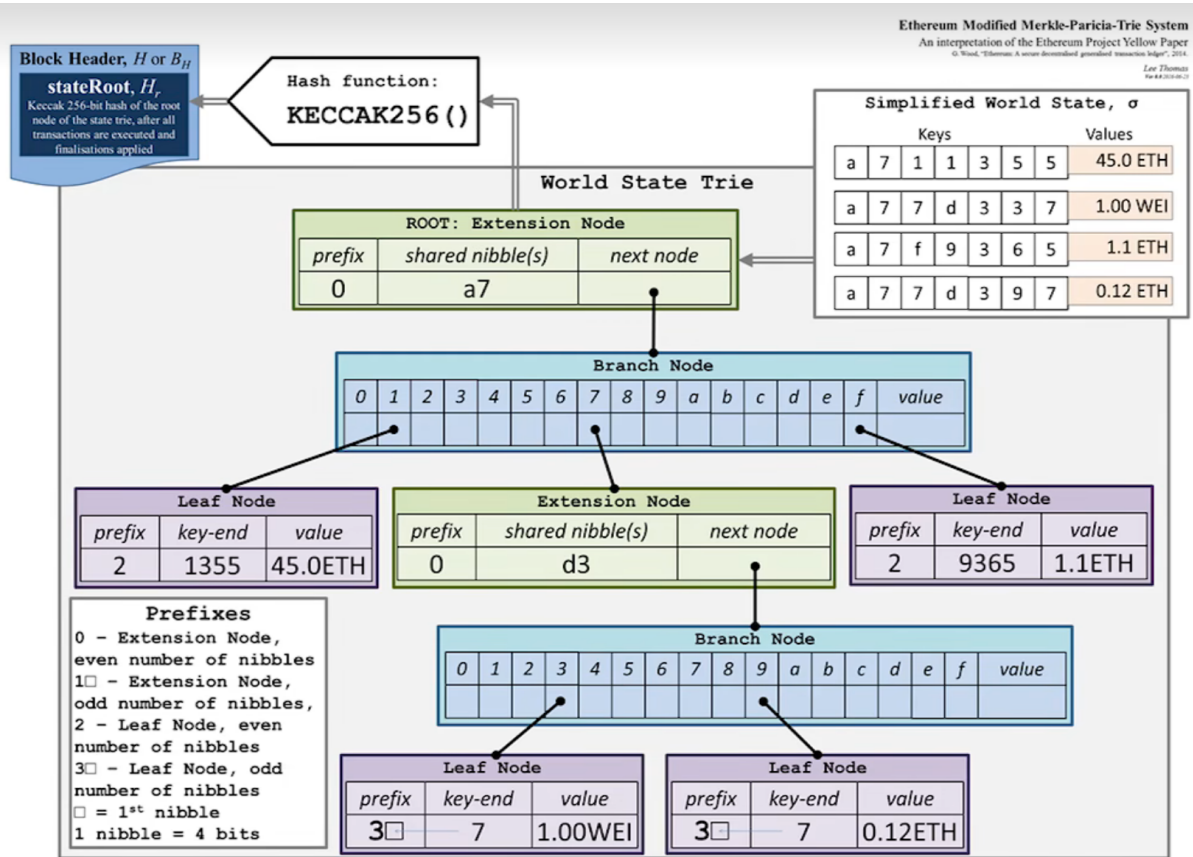


答曰：**节点密度低的时候**

节点密度低的时候，字典树的分叉会很少，此时的相同部分减少，个性部分增长，更适合这样缩减的数据结构。

因此，以太坊优化了字典树，设计出了下方的实例：

Patricia tree (trie) 压缩前缀树



数据结构

1. 共同节点 (extension node) 记录的是字符串共同的部分
2. 叶节点 (leaf node) 树的末端
3. 分支节点 (branch node)

数据结构可行的条件

在ETH账户体系中，最大的账户所有量是2的160次方，远远大于现有和可能有的账户数。

这种设计是为了避免哈希碰撞（不同地址统一账户）

也造成了账户的字符串分布稀疏的特点，适合这种数据结构

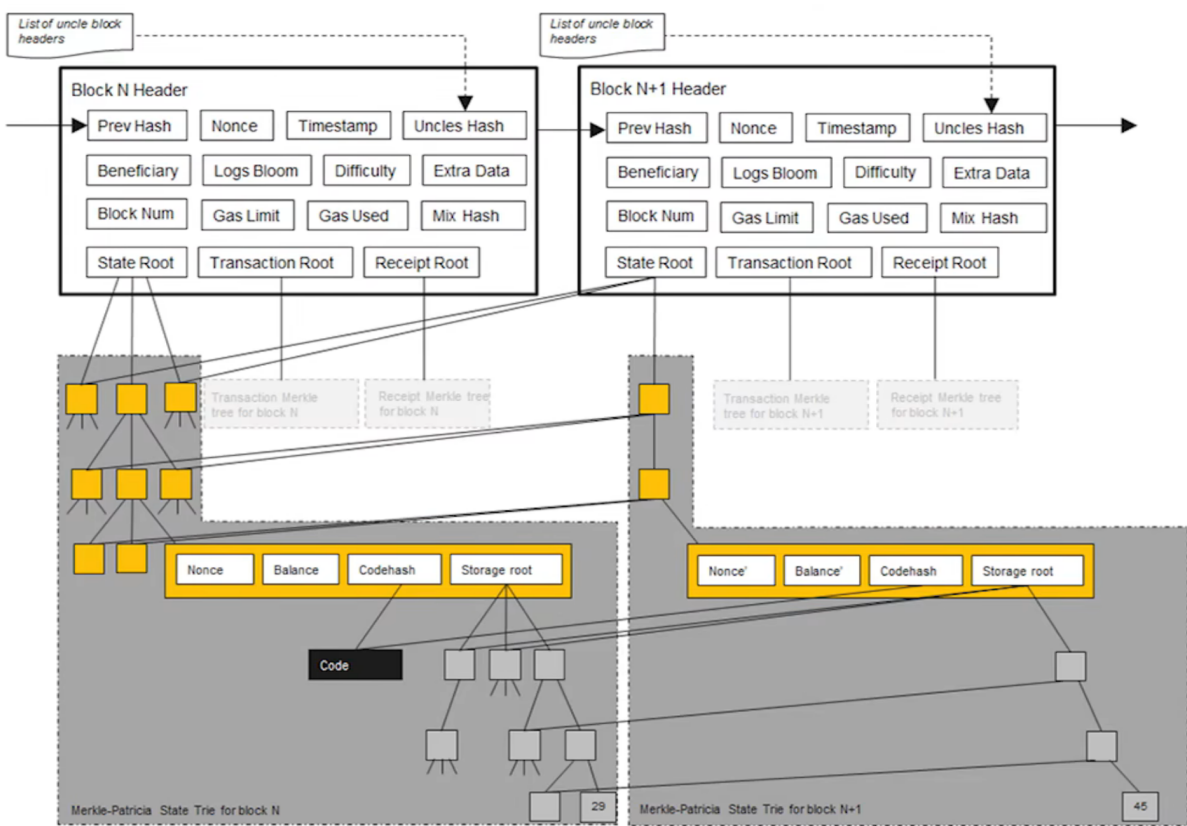
优点

1. 压缩了存储的40位哈希值的空间，降低了树的高度

MPT

与树和默尔克树，链表与哈希表一样。MPT (Merkle Patricia Tree) 就是将上面的指针同哈希指针替换形成的。

如何更新数据



- 1. 当新的交易产生时，状态树将变动的节点组成新的分支其余不变的分支点依旧对应原来的。
- 2. 当一个新的分支被创建时，通常是因为合约的状态发生了改变，导致新的状态需要被记录下来。对于老分支，由于状态已经更新，因此通常不再被使用。以太坊的客户端会自动处理这些老分支，将它们标记为无效或者清除，以便节省存储空间，并保持区块链的整洁。这些过程是由以太坊节点软件的内部机制来管理的，普通用户不需要手动处理老分支。
- 3. 一个状态树，包含很多小的MPT

为什么要保留？

方便回滚原来的状态。

相交于BTC的简单脚本，ETH的智能合约语法更为复杂，使得推测原先状态变得比较困难，因此要保留一段时间的上个状态的信息。

关于状态value的管理方式

上面的数据结构设计主要是针对（key，value）中key的查找。下面我们来分析一下value的查找方式

对于value的数据，一般经过RLP序列化处理之后才存储

RLP (Recursive Length Prefix)

极简主义序列化

只支持字节数组（nested array of bytes）

以太坊中的其他数据类型，都要变成字节数组之后进行序列化处理。之后有时间详细了解一下

实际数据结构

Block_Header的数据结构

```
69 // Header represents a block header in the Ethereum blockchain.
70 type Header struct {
71     ParentHash common.Hash    `json:"parentHash"      gencodec:"required"`
72     UncleHash   common.Hash    `json:"sha3Uncles"      gencodec:"required"`
73     Coinbase    common.Address `json:"miner"           gencodec:"required"`
74     Root        common.Hash    `json:"stateRoot"       gencodec:"required"`
75     TxHash      common.Hash    `json:"transactionsRoot" gencodec:"required"`
76     ReceiptHash common.Hash    `json:"receiptsRoot"    gencodec:"required"`
77     Bloom       Bloom          `json:"logsBloom"       gencodec:"required"`
78     Difficulty  *big.Int       `json:"difficulty"      gencodec:"required"`
79     Number      *big.Int       `json:"number"          gencodec:"required"`
80     GasLimit    uint64         `json:"gasLimit"        gencodec:"required"`
81     GasUsed     uint64         `json:"gasUsed"         gencodec:"required"`
82     Time        *big.Int       `json:"timestamp"       gencodec:"required"`
83     Extra       []byte         `json:"extraData"       gencodec:"required"`
84     MixDigest   common.Hash    `json:"mixHash"         gencodec:"required"`
85     Nonce       BlockNonce     `json:"nonce"           gencodec:"required"`
86 }
```

71. 前一个区块，区块头的哈希值

72. 叔父区块的地址（goast里面会讲）

73. 挖出区块矿工的地址

74. 状态树根哈希

75. 交易树的根哈希

76. 收据树的根哈希

77. 布隆过滤器（和收据树相关便于查询某种交易）

78. 挖矿难度

79. 用于数值计算

80. 智能合约部分讲

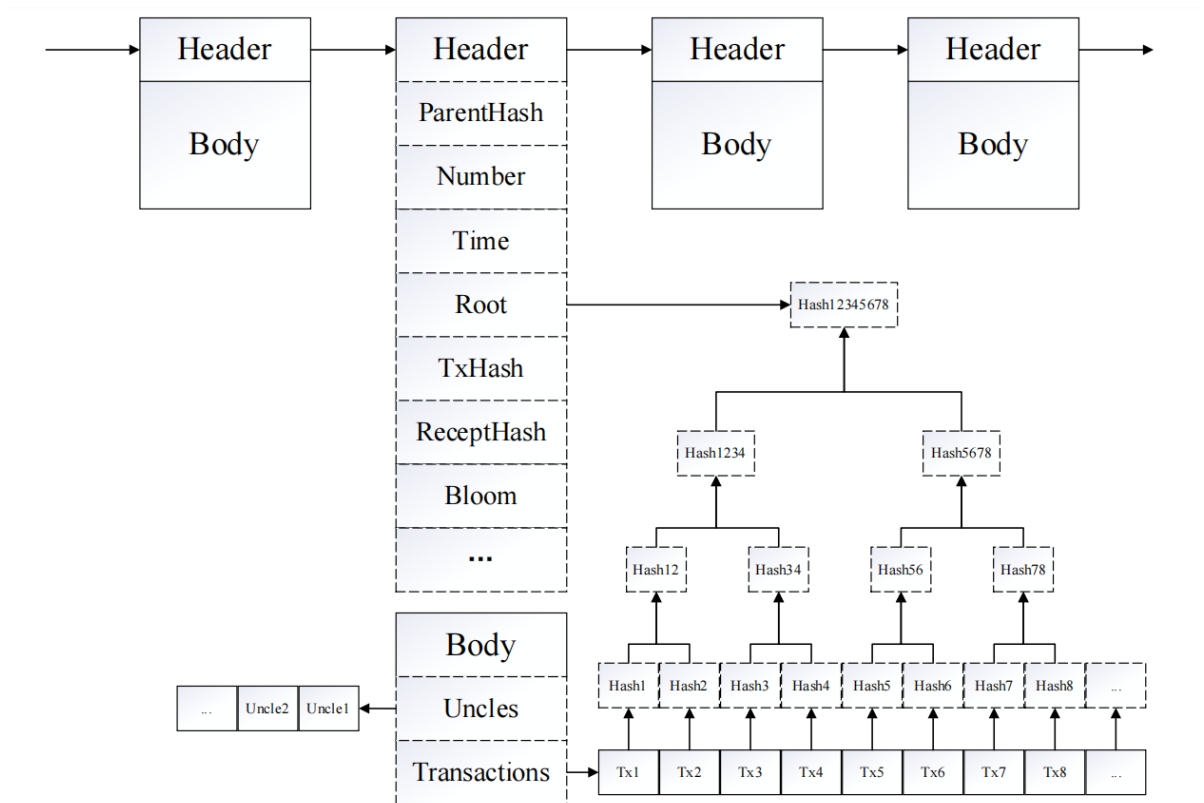
81. 智能合约部分讲

82. 大致产生时间

83. 挖矿部分讲

84. 挖矿部分讲

85. 随机数（最后找到的那个），和BTC里面的一样



图片来源：谷苏港《基于侧链的链上链下数据协同研究》论文

Block的数据结构

```

144 // Block represents an entire block in the Ethereum blockchain.
145 type Block struct {
146     header      *Header
147     uncles       []*Header
148     transactions Transactions
149
150     // caches
151     hash atomic.Value
152     size atomic.Value
153
154     // Td is used by package core to store the total difficulty
155     // of the chain up to and including the block.
156     td *big.Int
157
158     // These fields are used by package eth to track
159     // inter-peer block relay.
160     ReceivedAt time.Time
161     ReceivedFrom interface{}
162 }

```

146. 指向本区块Block_Header的指针

147. 指向叔父区块的Block_Header的指针 是一个数组，因为一个区块可以有多个叔父区块

148. 交易列表

实际发布的数据结构

真正在以太坊上面发布出去的只有上述的三个信息

```
177 // "external" block encoding. used for eth protocol, etc.
178 type extblock struct {
179     Header *Header
180     Txns    []*Transaction
181     Uncles  []*Header
182 }
```

交易树和收据树

数据类型

交易树与收据树都是基于Merkle tree（类似于BTC中的UTXO）的数据结构

实际上的这两种树和状态树一样，都是MPT

这样的设计可能是处于两点考虑：其一，三树代码统一，便于管理；其二，这样的结构便于查找（相较默尔克树）

两者的根哈希值保存在区块头中

这两种树存在的意义

1. 提供MerProof
2. 便于快速查找相关交易的结果（比如调用A合约的交易，或者某种类型的交易）

为了快速查找，以太坊维护了如下的一种数据类型（bloom filter）

bloom filter

这种数据类型是为 **高效查找某个集合中是否有某个元素**而生的

数据类型和原理

1. 将一个大集合整合成一个简短的摘要
2. 摘要的所有单元初始化均为0
3. 如果元素映射到摘要中，则该摘要变成1（只要求有映射，几个都一样）

数据类型特点及用途

1. 想查找a元素是否存在，求a元素映射到该摘要中的位置。如果该位置为0，则不存在；如果为1，则不一定存在
2. 该数据类型有：false positive的特性（能判断某元素不存在）
3. 没有 false negative的特性（不能判断某个元素是否存在）。因为可能会有哈希碰撞的产生（如元素B对应的映射也是a的这个位置）

使用bloom filter查找

1. 首先，这个摘要包含所有交易，位于区块头。
2. 我们求出想找交易A的映射，看看每个区块交易头中交易A的映射位置是否为0
 1. 如果为0，则说明该区块没有这种交易，跳过
 2. 如果为1，这说明可能有，遍历该区块中的交易树
3. 以太坊要bloom filter的目的就是为了排除肯定不存在该交易的区块，加速遍历

交易树收据树与状态树的区别

状态树 记录系统所有的账户地址 仅发生变化的新建分支

交易树收据树 记录本区块的交易对应的序号

对于以太坊的状态机的理解

以太坊可以看作是交易驱动的状态机

也就是，交易驱动这以太坊上的账户信息从一个状态走向下一个状态

状态的转移是确定且一致的，就像BTC里面的UTXO模型一样

为什么状态树不能一个区块一个？

1. 不好查找一个账户的信息（必须找到它的上一个交易）
2. 对于新创建的账户B，如果有A->B，则找到创世纪块才能确定B账户是新创建的

代码实现

创建状态树

```
package main

import (
    "github.com/ethereum/go-ethereum/core"
    "github.com/ethereum/go-ethereum/core/state"
    "github.com/ethereum/go-ethereum/core/types"
    "github.com/ethereum/go-ethereum/ethdb"
)

func createEthereumStateTree(db ethdb.Database, block *types.Block)
(*state.StateDB, error) {
    // Create a new state tree based on the parent block's state root
    parentState, err := state.New(block.ParentHash(), core.NewDatabase(db))
    if err != nil {
        return nil, err
    }

    // Create a new state tree based on the current block's transactions
    stateTree, err := state.New(block.Root(), core.NewDatabase(db))
    if err != nil {
        return nil, err
    }
}
```

```

    // Apply the transactions from the current block to the state tree
    for _, tx := range block.Transactions() {
        receipt, err := core.ApplyTransaction(parentState, stateTree, nil,
block.Header(), tx, &block.GasLimit, core.BlockChainConfig{})
        if err != nil {
            return nil, err
        }
        if receipt.Status != types.ReceiptStatusSuccessful {
            return nil, fmt.Errorf("transaction failed: %s", receipt.String())
        }
    }

    return stateTree, nil
}

```

创建交易树

```

package main

import (
    "github.com/ethereum/go-ethereum/core/types"
    "github.com/ethereum/go-ethereum/ethdb"
    "github.com/ethereum/go-ethereum/trie"
)

func createEthereumTransactionTree(db ethdb.Database, block *types.Block)
(*trie.Trie, error) {
    // Create a new trie for storing the transactions
    transactionTrie, err := trie.New(types.EmptyRootHash, trie.NewDatabase(db))
    if err != nil {
        return nil, err
    }

    // Insert the transactions into the trie
    for i, tx := range block.Transactions() {
        indexBytes := types.U256Bytes(new(big.Int).SetUint64(uint64(i)))
        err = transactionTrie.Update(indexBytes, tx.Hash().Bytes())
        if err != nil {
            return nil, err
        }
    }

    return transactionTrie, nil
}

```

创建收据树

```

package main

import (
    "github.com/ethereum/go-ethereum/core/types"
    "github.com/ethereum/go-ethereum/ethdb"

```

```
"github.com/ethereum/go-ethereum/trie"
)

func createEthereumReceiptTree(db ethdb.Database, block *types.Block)
(*trie.Trie, error) {
    // Create a new trie for storing the receipts
    receiptTrie, err := trie.New(types.EmptyRootHash, trie.NewDatabase(db))
    if err != nil {
        return nil, err
    }

    // Insert the receipts into the trie
    for i, receipt := range block.Receipts() {
        indexBytes := types.U256Bytes(new(big.Int).SetUint64(uint64(i)))
        rlpBytes, err := rlp.EncodeToBytes(receipt)
        if err != nil {
            return nil, err
        }
        err = receiptTrie.Update(indexBytes, rlpBytes)
        if err != nil {
            return nil, err
        }
    }

    return receiptTrie, nil
}
```